



Análisis de datos ómicos PEC 1

Juan Carlos Forero Niampira

<https://github.com/juanforero15/Forero-Niampira-Juan-Carlos-PEC1.git>

Contents

Resumen	1
Objetivos	1
Metodos	1
Resultados	2
Creación contenedor	2
Análisis exploratorio	3
Distribución muestra - individuos	6
Trasformación datos	7
Análisis de componentes principales	11
Discusión	15
Referencias	16

```
dir.create("datos")
```

Resumen

En esta PEC se realiza una introducción a las ómicas mediante un ejercicio que permite trabajar con algunas de las herramientas como Bioconductor y la exploración multivariante de datos. Para llevar a cabo esto, es necesario estar relacionado con las tecnologías ómicas, Bioconductor y las clases que se utilizan para almacenar datos ómicos, como los expressionSets y en este caso de Summarizedexperiment, Git como herramienta de control de versiones, así como GitHub.

Objetivos

El objetivo de esta PEC es que planifiquéis y ejecutéis una versión simplificada del proceso de análisis de datos ómicos, a la vez que practicáis con algunas de las herramientas y métodos que hemos trabajado durante el primer reto.

Metodos

Diferenciamos básicamente dos grupos de métodos. Los métodos bioinformáticos y los estadísticos. El principal método bioinformático que se utiliza es la creación de clases contenedoras para datos de expre-

sión, ExpressionSets. Los recursos para la solución de la PEC son los que se han proporcionado en el aula. Adicionalmente se realiza en este documento un rápido aprendizaje del uso de SummarizedExperiment en vez de ExpressionSet, siguiendo como base el tutorial disponible en Bioconductor (<https://bioconductor.org/packages/release/bioc/vignettes/SummarizedExperiment/inst/doc/SummarizedExperiment.html>).

En cuanto a los métodos estadísticos podemos diferenciar análisis uni y bivalente de los datos, mediante boxplots y/o histogramas para estudiar la forma general de los mismos y mediante correlogramas y matrices de distancias para estudiar la relación entre variables e individuos.

Análisis multivalente de los datos, mediante Análisis de Componentes Principales (PCA) y Agrupamiento Jerárquico, para determinar si los grupos que aparezcan (en caso de hacerlo) parecen relacionarse con las fuentes de variabilidad del estudio o, si por el contrario, podrían haber otras fuentes de variabilidad como efectos batch.

Los datos se seleccionaron y descargaron desde el repositorio de GitHub (<https://github.com/nutrimetabolomics/metaboData/tree/main/Datasets/2018-MetabotypingPaper>), seleccionado el dataset de 2018-MetabotypingPaper.

Finalmente este documento se conectó con el repositorio con el mismo nombre, y actualizado mediante la herramienta “push” y tras previo enlace al momento de crear este proyecto y enlazado en el siguiente link: <https://github.com/juanforero15/Forero-Niampira-Juan-Carlos-PEC1.git>

Resultados

En primera medida se procede a cargar los archivos, con lo cual se observa que el número de columnas de datos coinciden con el número de filas de metadatos. Por otro lado debido a que se requiere que los datos estén en matrices numéricas, se hace necesario en este caso que al dataframe Datos en las primeras 5 columnas contienen información sobre los individuos. Dichos datos se suelen asignar a rowData, para lo cual se separaron los datos en 2 dataframes.

```
metaDatos <- read.csv("datos/DataInfo_S013.csv", row.names = 1)
datos <- read.csv("datos/DataValues_S013.csv", row.names = 1)
class(datos)
```

```
## [1] "data.frame"
```

```
infoSamples <- datos[, 1:5]
if (ncol(datos) == 695) datos <- datos[, -c(1:5)]
if (nrow(metaDatos) == 695) metaDatos <- metaDatos[-c(1:5), ]
```

Creación contenedor

Mediante la clase SummarizedExperiment se genera un contenedor para los datos omicos, para lo cual una vez instalado el paquete se procede a crear el contenedor. Se utilizarán 3 bloques de información siendo los datos que irán en el rowData, la información de las variables que irán en colData. Así pues, se convierten los datos e información de las muestras a un tipo adecuado de datos, asignando como factores el nombre de los objetos infoSamples con nombres informativos de las filas.

```
if (!require(BiocManager)) install.packages("BiocManager")
if (!require(SummarizedExperiment)) BiocManager::install("SummarizedExperiment")

library(dplyr)
# Convertir las columnas 2 (SURGERY), 4 (GENDER) y 5 (Group) a factores
infoSamples <- infoSamples %>%
mutate(SURGERY = as.factor(SURGERY), GENDER = as.factor(GENDER), Group = as.factor(Group))
# Crear el nuevo nombre para las filas
infoSamples <- infoSamples %>%
mutate(RowName = paste0(toupper(substr(SURGERY, 1, 1)), "_", Group, "_", SUBJECTS))
```

```
rownames(infoSamples) <- infoSamples$RowName
infoSamples <- infoSamples %>%
select(-RowName)
head(infoSamples)
```

```
##      SUBJECTS SURGERY AGE GENDER Group
## B_1_1         1 by pass  27      F      1
## B_2_2         2 by pass  19      F      2
## B_1_3         3 by pass  42      F      1
## B_2_4         4 by pass  37      F      2
## T_1_5         5 tubular  42      F      1
## B_2_6         6 by pass  24      F      2
```

Posteriormente se procede de forma parecida con los datos , convirtiendo la matriz numérica nombrando las fila igual que las fila de infoSamples. Seguido de esto se crea la clase SummarizedExperiment.

```
matDatos <- as.matrix(datos)
if (sum(rownames(matDatos) != infoSamples$SUBJECTS) == 0) rownames(matDatos) <- rownames(infoSamples)

mySE <- SummarizedExperiment(assays = list(rawValues = matDatos), rowData = infoSamples,
colData = metaDatos)
show(mySE)
```

```
## class: SummarizedExperiment
## dim: 39 690
## metadata(0):
## assays(1): rawValues
## rownames(39): B_1_1 B_2_2 ... T_1_38 B_1_39
## rowData names(5): SUBJECTS SURGERY AGE GENDER Group
## colnames(690): MEDDM_TO MEDCOL_TO ... SM.C24.0_T5 SM.C24.1_T5
## colData names(3): VarName varTpe Description
```

Los metadatos (rowData y colData) pueden variar o incluso faltar entre diferentes experimentos debido a que cada experimento es único y puede requerir información complementaria distinta a lo cual se suma la falta de estandarización para definir metadatos mínimos lo que genera inconsistencias entre estudios.

Por lo tanto, la diversidad experimental y la ausencia de normas claras hacen que los metadatos puedan ser heterogéneos o incompletos.

Analisis exploratorio

Metadatos

En esta exploración se proporciona información de posible grupos, en este caso los datos no son balanceados pero hay de cierto modo una repartición de grupos.

```
data <- rowData(mySE)[, c("SURGERY", "AGE", "GENDER", "Group")]
# Convertir a data.frame para facilidad de manipulación
data <- as.data.frame(data)
# Calcular tablas de frecuencias para las columnas 1 (SURGERY), 3 (GENDER) y 4
# (Group)
(freq_surgery <- table(data$SURGERY))

##
## by pass tubular
##      26      13
```

```

(freq_gender <- table(data$GENDER))

##
## F M
## 27 12

(freq_group <- table(data$Group))

##
## 1 2
## 24 15

(freq_surgery_Group <- table(data$SURGERY, data$Group))

##
##          1  2
## by pass 13 13
## tubular 11  2

(freq_surgery_Group_Gender <- table(data$SURGERY, data$Group, data$GENDER))

## , , = F
##
##
##          1  2
## by pass 10 10
## tubular  6  1
##
## , , = M
##
##
##          1  2
## by pass  3  3
## tubular  5  1

# Calcular el resumen numérico para la columna 2 (AGE)
(summary_age <- summary(data$AGE))

##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 19.00   35.00   41.00   40.79   46.00   59.00

```

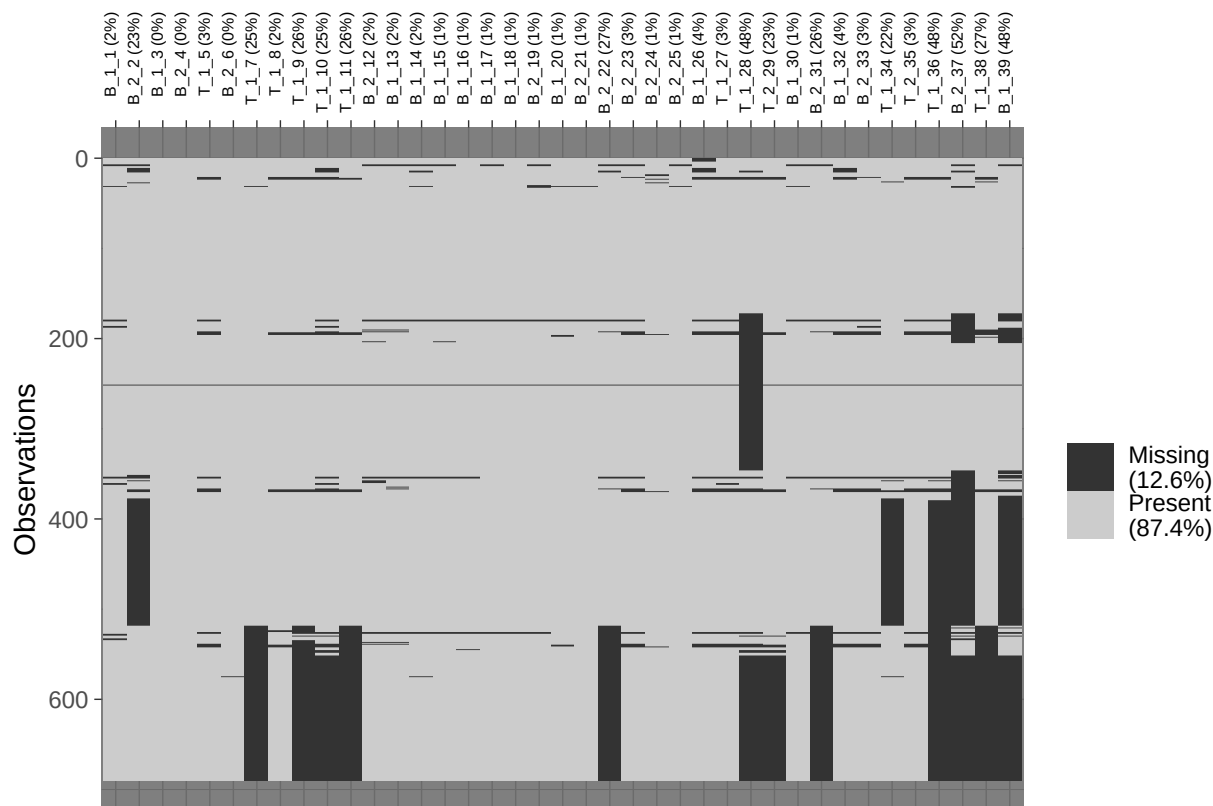
Seguido de este paso, es importante realizar una identificación y clasificación de los valores faltantes para poder realizar e interpretar determinados análisis, con lo cual se asigna un valor mínimo teniendo en cuenta que se asume dichos valores no están bien sea porque no se detectaron o están ausentes. Este proceso se puede realizar utilizando el paquete `nanian` mediante la transformación en dataframe de la matriz de `assay(mySE"rawValues")`, adicionalmente se pueden graficar las etiquetas que en este caso se ve un alto porcentaje de valores faltantes, lo cual se puede confirmar también mediante valores numéricos.

```

if (!require(nanian)) install.packages("nanian")
library(nanian)
# Visualización de valores faltantes
dataf <- as.data.frame(t(assay(mySE, "rawValues")))
# rownames(dataf) <- rownames(rawVals) colnames(dataf) <- colnames(rawVals)
library(ggplot2)
# Visualización con ajuste del tamaño de etiquetas
vis_miss(dataf) +
  theme_dark() +

```

```
theme(axis.text.x = element_text(size = 6, angle = 90, hjust = 1, color = "black"))
```



```
# valore numericos de valores faltantes
```

```
n_miss(dataf)
```

```
## [1] 3390
```

```
n_complete(dataf)
```

```
## [1] 23520
```

```
prop_miss(dataf)
```

```
## [1] 0.1259755
```

```
prop_complete(dataf)
```

```
## [1] 0.8740245
```

```
pct_miss(dataf)
```

```
## [1] 12.59755
```

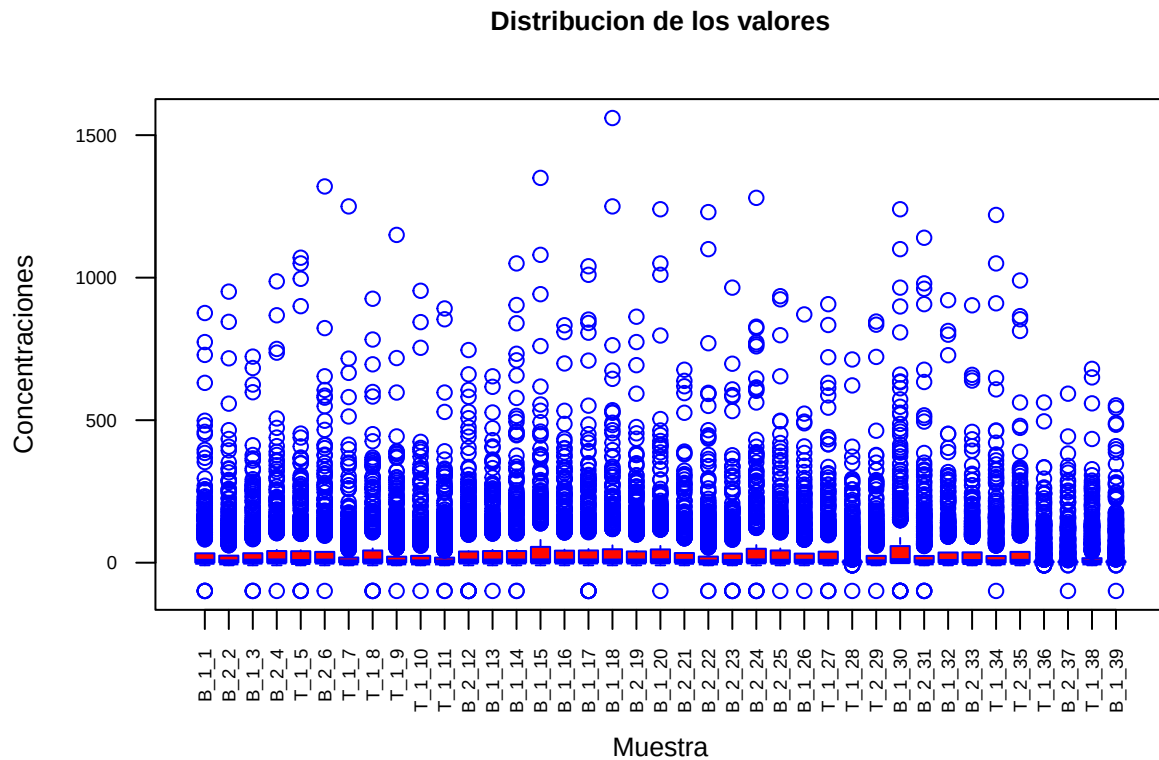
```
pct_complete(dataf)
```

```
## [1] 87.40245
```

En los valores que se obtienen en metabolómica no se suelen detectar algunos valores, lo cual se soluciona asignando un 0 o 1.

```
## class: SummarizedExperiment
## dim: 39 690
## metadata(0):
## assays(2): rawValues rawNoMissings
## rownames(39): B_1_1 B_2_2 ... T_1_38 B_1_39
## rowData names(5): SUBJECTS SURGERY AGE GENDER Group
## colnames(690): MEDDM_T0 MEDCOL_T0 ... SM.C24.0_T5 SM.C24.1_T5
## colData names(3): VarName varTpe Description
```

```
boxplot(t(assay(mySE, "rawNoMissings")), col=heat.colors(ncol(assay(mySE))), xlab = "Muestra", ylab = "Concentraciones")
```



Trasformación datos

Debido a la alta variación es posible simetrizar los datos mediante logaritmos y el cual puede ser aplicado en el contendor.

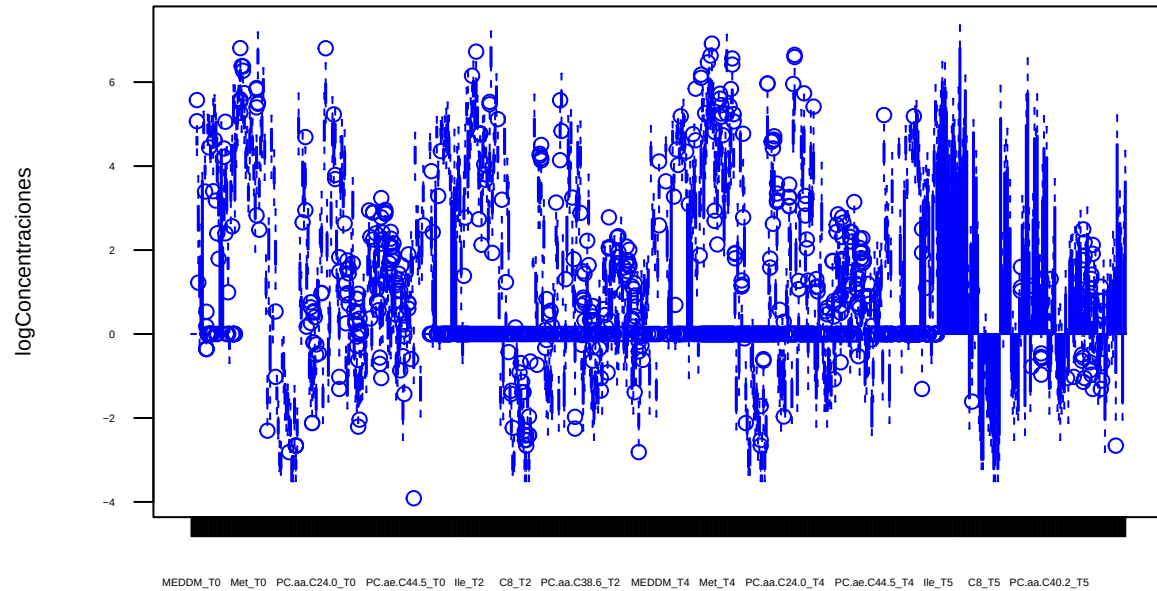
```
logNoMissings <- log(assay(mySE, "rawNoMissings"))
assays(mySE)$logNoMissings <- logNoMissings
show(mySE)
```

```
## class: SummarizedExperiment
## dim: 39 690
## metadata(0):
## assays(3): rawValues rawNoMissings logNoMissings
## rownames(39): B_1_1 B_2_2 ... T_1_38 B_1_39
## rowData names(5): SUBJECTS SURGERY AGE GENDER Group
## colnames(690): MEDDM_T0 MEDCOL_T0 ... SM.C24.0_T5 SM.C24.1_T5
## colData names(3): VarName varTpe Description
```

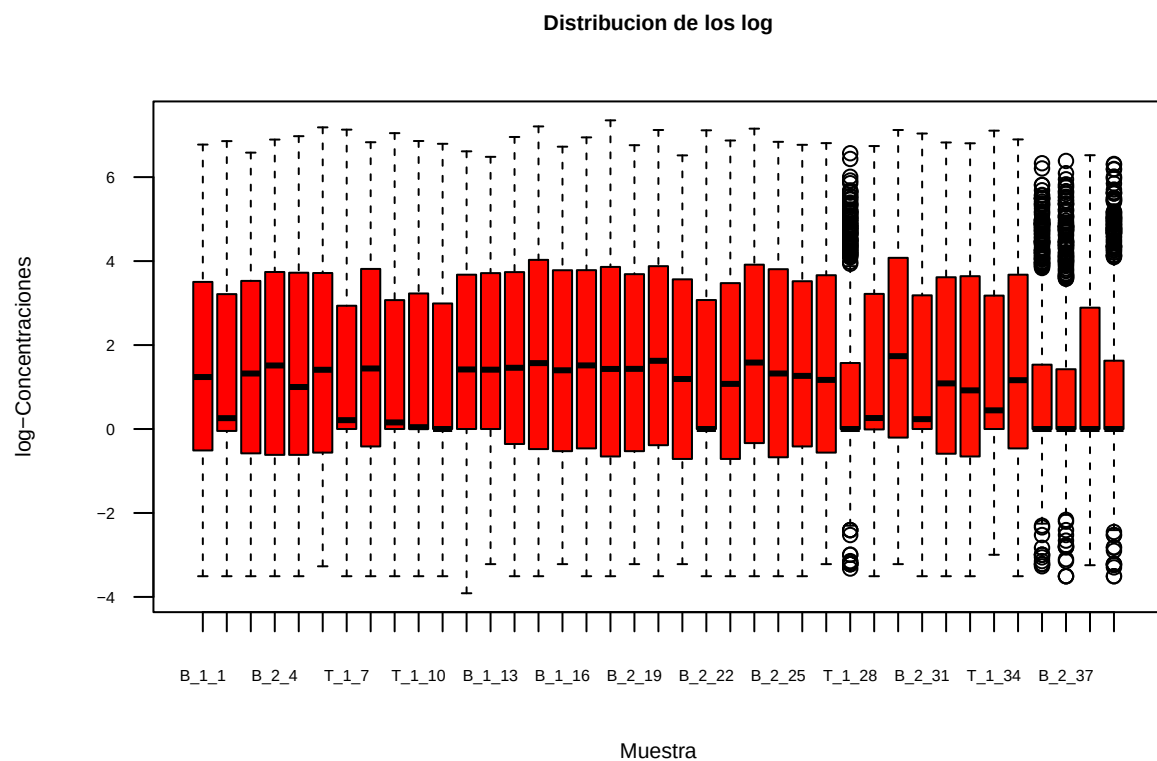
Para verificar la diferencia despues de la trasformación, que en este caso tienen un efecto positivo en cuanto a la simetria sin embargo es importante tener en cuenta aquellas muestras con valores perdidos que tienen los peores resultados.

```
boxplot(assay(mySE, "logNoMissings"), ylab = "logConcentraciones", cex.lab = 0.7,
horizontal = FALSE, cex.axis = 0.3, las = 1, main = "Distribucion de los logvalores por metabolito",
cex.main = 0.8, border = "blue")
```

Distribucion de los logvalores por metabolito



```
boxplot(t(assay(mySE, "logNoMissings")), col=heat.colors(ncol(assay(mySE))), xlab = "Muestra", ylab = "logConcentraciones",
cex.lab = 0.7, horizontal = FALSE, cex.axis = 0.5, las = 1, main = "Distribucion de los log", cex.main = 1.2)
```



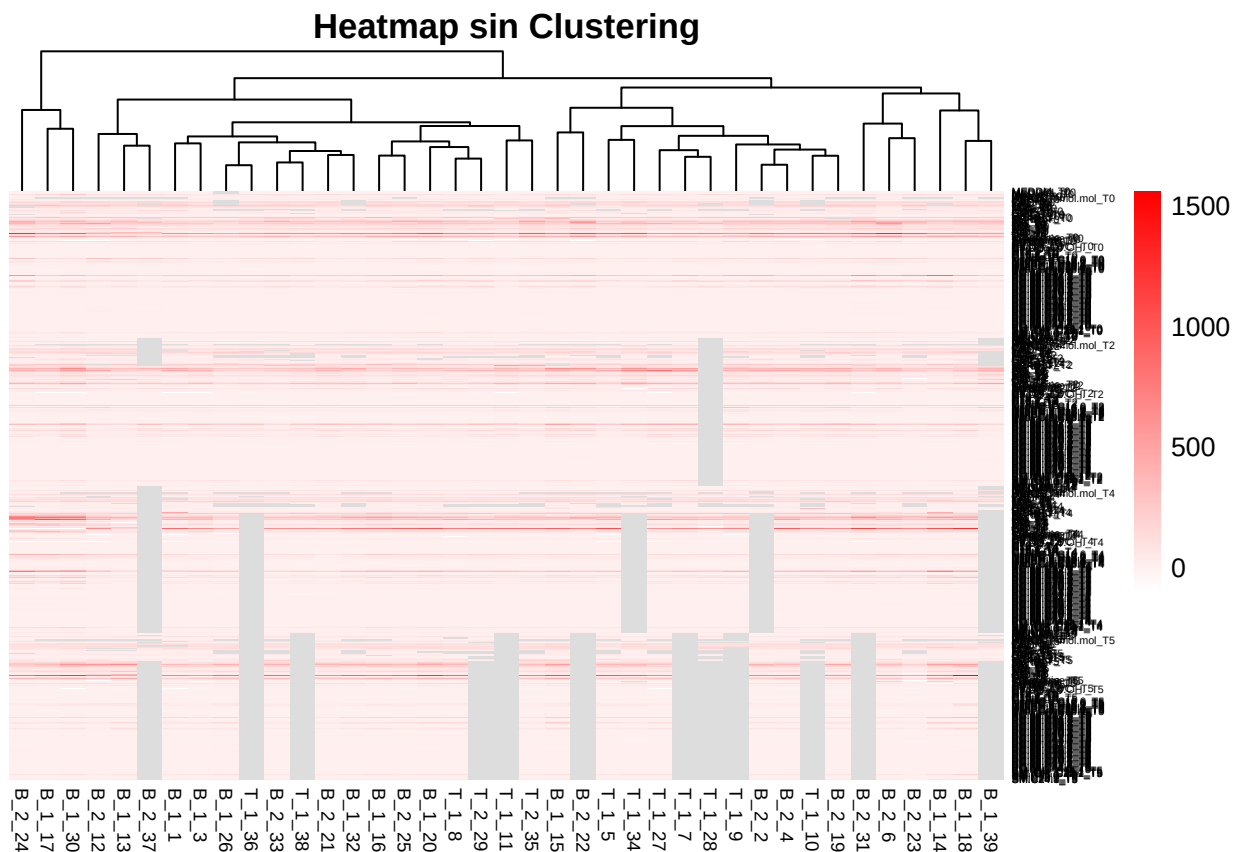
Visualización individuos/variables

Una forma eficiente de visualizar los datos es mediante un mapa de calor

```
if (!require(pheatmap)) install.packages("pheatmap")
library(pheatmap)
# Heatmap sin clustering
pheatmap(t(assay(mySE, "rawNoMissings")), cluster_rows = FALSE, cluster_cols = FALSE, main = "Heatmap s
```

```
# Heatmap sin clustering
```

```
heatmap(t(assay(mySE, "rawValues")), cluster_rows = FALSE, cluster_cols = TRUE, main = "Heatmap sin Cl
```



Análisis de componentes principales

Para realizar este análisis se empleará el paquete PCAtools para lo cual se requiere un dataframe con datos numéricos y otra con metadatos. Se extrae la matriz de datos de rawNoMissings del objeto SummarizedExperiment (mySE), que contiene los datos de expresión. Posteriormente se transpone la matriz intercambiando filas por columnas). Se espera que las filas = muestras y que las columnas = variables (genes, metabolitos...)

```
if (!require(PCAtools)) BiocManager::install("PCAtools", dep = TRUE)
```

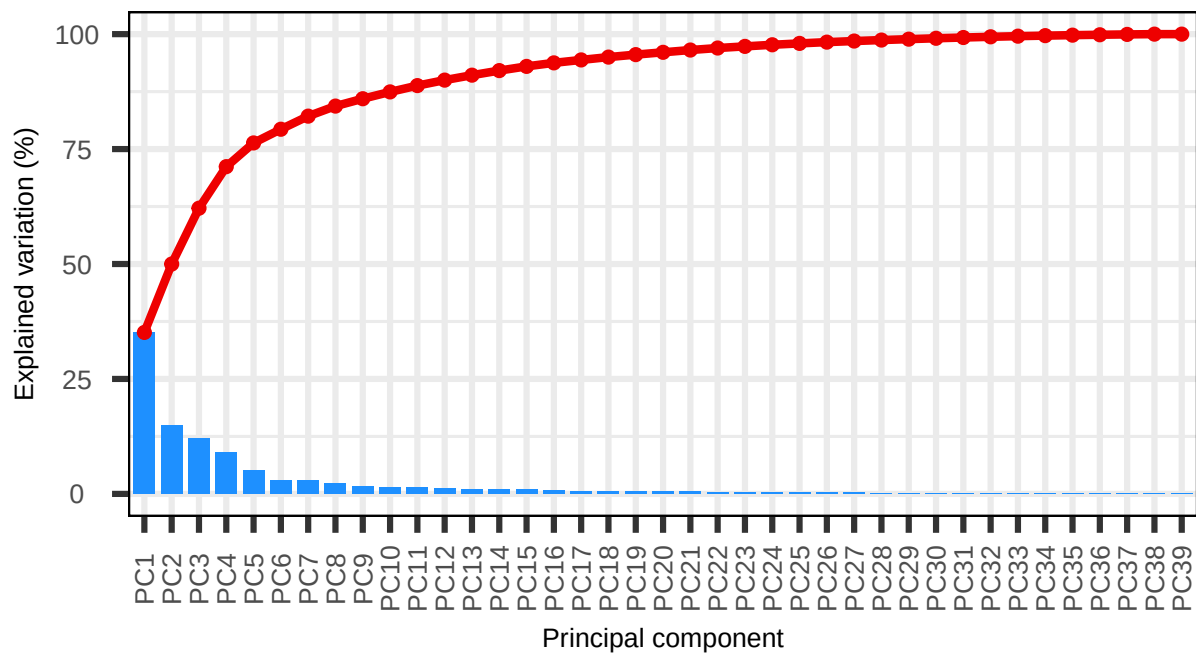
```
library(PCAtools)
matriz <- t(assay(mySE, "rawNoMissings"))
pclas <- pca(matriz, metadata = rowData(mySE), removeVar = 0.1)
class(pclas)
```

```
## [1] "pca"
```

Podemos explorar los datos utilizando el objeto pclas como base, siguiendo las indicaciones de la viñeta del paquete.

Para esto se evalúa la importancia de cada componente principal mediante la función screeplot(), la cual visualiza el porcentaje de variabilidad que explica cada componente.

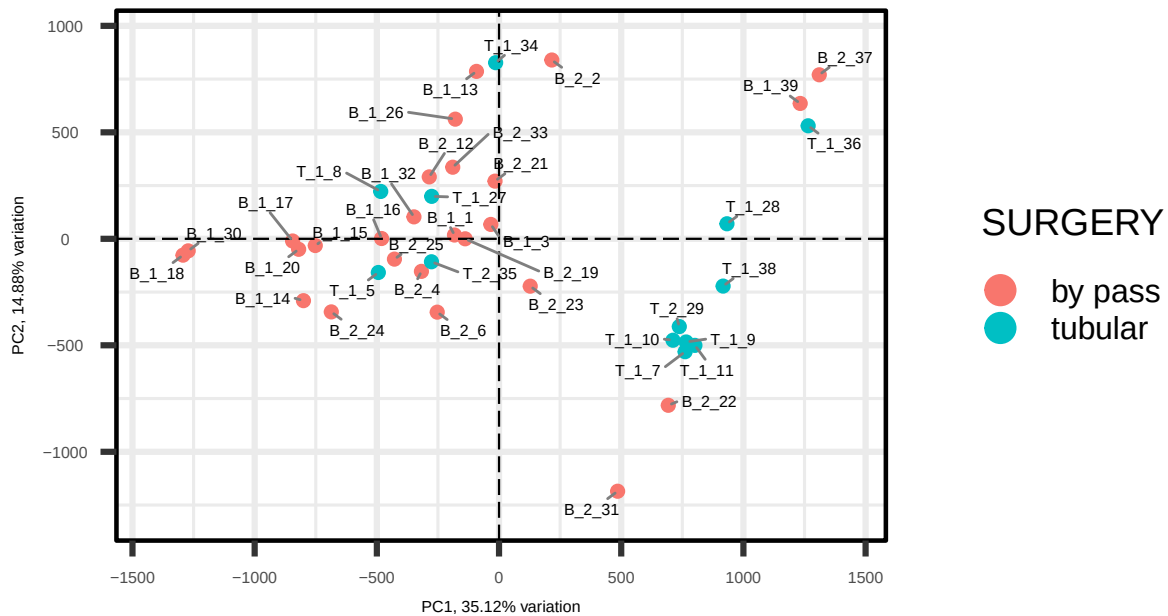
```
screeplot(pclas, axisLabSize = 10, titleLabSize = 1)
```



El gráfico revela que los dos primeros componentes principales capturan aproximadamente el 50% de la variabilidad total de los datos. Esto indica que las 2-3 primeras componentes son adecuadas para representar la estructura fundamental de los datos y que proporcionan una visión suficientemente informativa sin necesidad de considerar componentes adicionales.

```
biplot(pclas, showLoadings = FALSE, labSize = 2, pointSize = 2, axisLabSize = 6, title = "PCA datos" , s
```

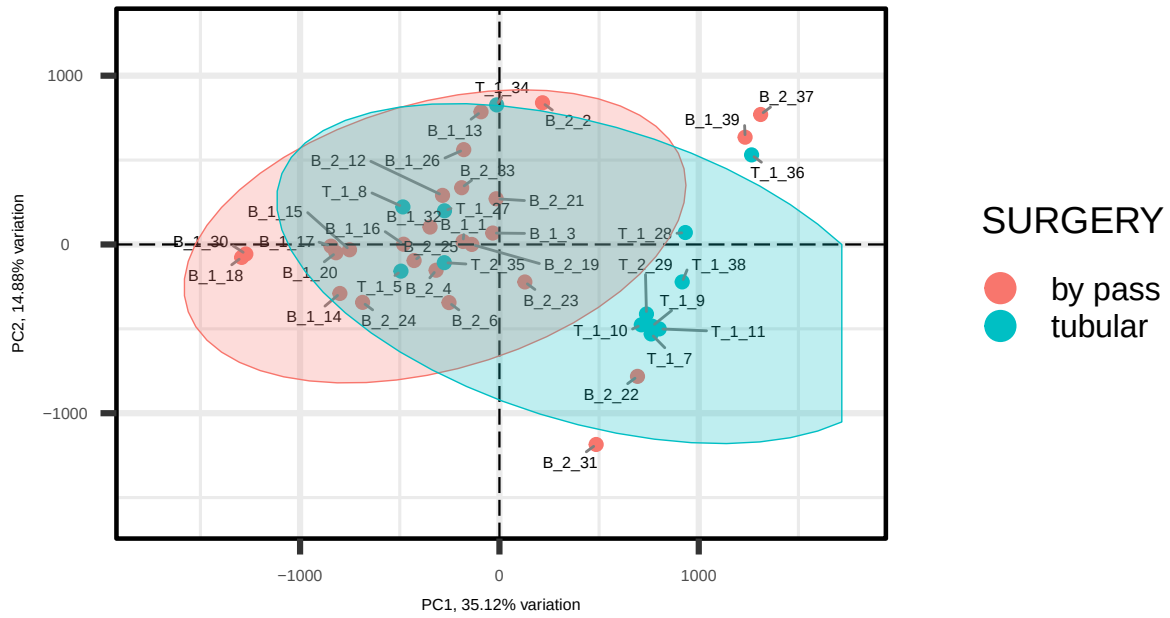
PCA datos



En este gráfico se muestra que al diferenciar las muestras según la variable “SURGERY” incluida en los metadatos, se aprecia cierta distinción a lo largo del primer componente principal (PC1). Sin embargo, esta separación no es completa, debido a que algunos puntos de diferentes grupos aparecen superpuestos. La tendencia se visualiza con mayor claridad cuando se añaden elipses de confianza alrededor de cada grupo, las cuales ayudan a delimitar los espacios que ocupan mayoritariamente las muestras de cada categoría.

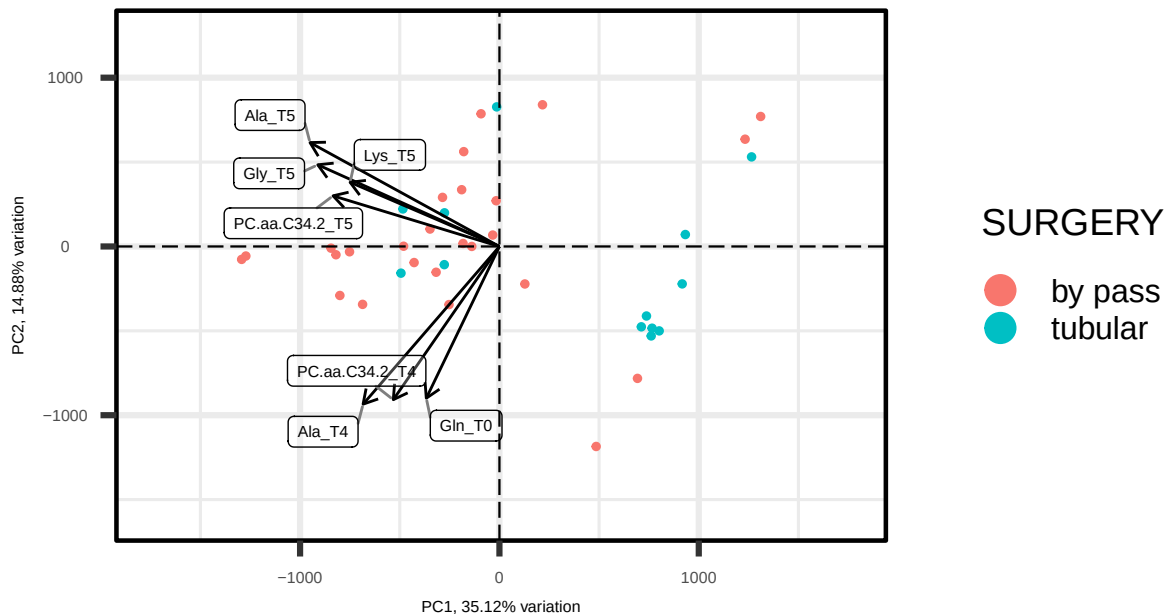
```
biplot(pclas, showLoadings = FALSE, labSize = 2, pointSize = 2, axisLabSize = 6, ellipse = TRUE,
title = "PCA de los datos", sizeLoadingsNames = 3, colby = "SURGERY", hline = 0,
vline = 0, legendPosition = "right")
```

PCA de los datos



```
biplot(pclas, showLoadings = TRUE, labSize = 2, pointSize = 1, axisLabSize = 6, title = "PCA datos", si
```

PCA datos



Este análisis nos revela que diversos metabolitos muestran una clara asociación con valores extremos altos o bajos, específicamente en el segundo componente principal, pero no presentan esta misma relación con el primer componente. Esta distribución diferencial puede visualizarse mediante un gráfico de cargas loadings que muestra el peso específico de cada variable en la formación de los componentes principales, destacando metabolitos con mayor contribución a PC2 y la ausencia de esta misma influencia en PC1

Discusión

Calidad de Datos y Preprocesamiento.

En este análisis se tuvo la presencia de un 12.6% de valores faltantes en un dataset de metabolómica con 690 variables con lo cual, si los NA reflejan límites técnicos de detección, la imputación a un valor mínimo (1) es una aproximación aceptable pero subóptima. Se tendría que valorar otros métodos que preserven mejor la estructura estadística de los datos.

Respecto a la heterogeneidad observada podría estar inflada por efecto batch dado por lotes experimentales, días de procesamiento entre otros.

Transformación de Datos y Estabilidad de Resultados.

Dada la asimetría en los datos, aplicar una transformación logarítmica es una estrategia acertada aunque es importante valorar si existen valores cero o negativos que requieran ajustes.

Resultados del PCA.

La separación parcial por tipo de “SURGERY” puede entenderse a razón de respuestas metabólicas individuales a la cirugía moduladas por factores genéticos o epigenéticos o bien por la variabilidad introducida durante la adquisición de datos.

Referencias

<https://bioconductor.org/packages/release/bioc/vignettes/SummarizedExperiment/inst/doc/SummarizedExperiment.html>)

https://github.com/ASPteaching/Analisis_de_datos_omicos-Ejemplo_0-Microarrays

https://github.com/ASPteaching/Analisis_de_datos_omicos-Ejemplo_1-Microarrays

<https://www.metabolomicsworkbench.org/>

<https://www.bioconductor.org/packages/release/bioc/html/POMA.html>